# **COMP41790** – Foundations of Deep Learning Project

**Version 0.81 – Autumn 2025**

## Overview

This individual project assesses your ability to apply modern imaging and computer vision techniques to a real-world problem. The project contributes 60% of the final grade.

The goal is to design a vision pipeline that uses deep learning to solve a video classification problem, namely identifying tennis strokes as forehand, backhand, serve, or other, using a small dataset of video clips captured from the back of a tennis court. You will be required to implement and train a vision model, evaluate its performance, and present your findings in both a written report and a video presentation.

This project is intentionally open-ended to allow you to explore different methods and strategies. You are expected to research and apply state-of-the-art deep learning and computer vision approaches to achieve the best possible results.

---

## 1 Specification

The project is divided into three components: Code, Report, and Video Presentation.

### 1.1 Code

You will develop your project using **PyTorch (version $\geq$ 2.1.0)**, **Python (version $\geq$ 3.11.4)**, and **OpenCV (version $\geq$ 4.10.0)**. These are standard frameworks in modern computer vision, so becoming comfortable with them is an important learning outcome for this project. It is therefore recommended to start early in case you run into any problems!

Your code should be packaged as a **Jupyter notebook** with a set of Python scripts that are:

- **Clear:** logically structured and easy to read,
- **Commented:** explain the key parts of your implementation,
- **Reproducible:** all experiments must be fully reproducible.[1]

If you use external code (e.g., for preprocessing, pretrained models, or other utilities), you must clearly acknowledge the source within the code and include a proper citation or link in your report. You may freely use pretrained models to improve performance, but you should explain:

- the motivation for using a specific model, and
- why it is appropriate for the given problem.

It is also required that you first develop a **baseline model** trained from scratch (without fine-tuning or transfer learning). This will help you understand the problem, establish a performance benchmark, and provide a basis for comparison when experimenting with more advanced techniques.

### 1.2 Report

The written report should follow a **conference-style format**, using the NeurIPS LaTeX template (compiled as `preprint`) and style guide, available at: `https://neurips.cc/Conferences/2020/PaperInformation/StyleFiles`.

The report should be **4–6 pages in length** (excluding references) and include the following sections:

- **Abstract**
- **Introduction**
- **Related Work**
- **Experimental Setup**
- **Results**

[1] A brief guide to reproducible machine learning can be found at `https://machinelearningmastery.com/reproducible-machine-learning-results-by-default/`.

– **Conclusion & Future Work**
– **References**

A description of these sections can be found in the Resources section of this document.

### 1.3 Video Presentation

You are also required to create a short **5-minute video presentation** summarising your project. The goal is to clearly communicate your methodology, findings, and reflections.

Your presentation should include the following:

– *Introduction:* Introduce yourself (name and student number). Briefly describe the project topic, problem statement, and objectives.
– *Walkthrough:* Present your Jupyter notebook, highlighting key code sections and explaining your reasoning as you developed the solution. Execute key cells during the walkthrough.
– *Model Explanation:* Describe your model architecture, hyperparameters, and optimisation strategy. Explain why you selected this approach.
– *Training Insights:* Discuss any challenges or insights gained during model training, and how you addressed them.
– *Testing and Evaluation:* Demonstrate how you evaluated performance, including the metrics used and your interpretation of the results.
– *Conclusion:* Summarise your key findings and lessons learned. Suggest how the project could be extended or improved, and briefly reference your report.

Keep your video concise and engaging (**5 minutes maximum**). Record it using screen-sharing software such as **Zoom** or **Microsoft Teams**, ensuring your **face is visible** in a small embedded window. The final video must be encoded in **MP4 format** using the **H.264 codec**, which offers high quality and efficient compression.

---

## 2 Dataset

A small dataset will be provided for this project on a shared drive. It contains video clips of tennis players performing different strokes, captured from the back of a tennis court. Each clip is grouped according to the type of stroke being performed: *forehand*, *backhand*, *serve*, or *other*.

A portion of the data has be held back, should we need to check your model's final performance (you will not have access to these samples).

It is your responsibility to determine how best to process the video data for model input. Possible approaches include:

– Frame extraction and selection
– Using 3D convolutional neural networks to model spatio-temporal patterns
– Applying pose estimation models to extract keypoints as input features

Ensure that your dataset has a clearly defined **train/validation/test** split in your submission.

**Note:** given the limited size of the dataset, you should explore techniques such as data augmentation, transfer learning, and regularization to improve model performance.

**You are also encouraged to collaborate with your peers to annotate additional video clips** and/or generate synthetic data (e.g., using diffusion models) to enhance the dataset.

---

## 3 Experiments

### 3.1 Useful considerations before starting

– What pre-processing is necessary to perform before modelling data?
– What is the structure of the data? Is it sequential?
– Is the data balanced?
– If so, what techniques can be used to address this imbalance?
– What deep learning architectures are most appropriate for this data?
– Can the data be changed to fit a particular architecture?
– Are there existing pre-trained models that could be fine-tuned?
– Can the model be parallelised (make better use of GPUs for quicker training)?
– How many weights are there in the model? How much memory does this consume?

### 3.2 Evaluating model performance

A **validation set** should be created either by selecting a subset of the training set. This set is used to evaluate your model's ability to **generalise** and to perform **hyperparameter tuning**. **Never use the test set for this purpose.**

You should generate a plot with **training and validation loss** on the $y$-axis and training iterations on the $x$-axis to help identify the point at which the model generalises best. The model that achieves the best performance on the validation set should then be used for final testing.

Explain and justify your choice of **evaluation metrics**, ensuring they are appropriate for your dataset and task. Report the model's performance using these metrics.

The **training procedure may be repeated during grading**. While small variations in training loss due to random initialisation are expected, significant discrepancies between the reported and observed results may raise concerns about the validity of the experiment and, consequently, the overall project.

Additionally, you must **submit a baseline model** that is coded and trained from scratch, without fine-tuning. Your submitted code and report must clearly define and describe your model architecture. Furthermore, you should perform **hyperparameter tuning** and justify the **chosen model topology** based on your findings.

---

# 4 Submission Details

## 4.1 What to submit

You are required to submit 2 files: **(i) a zip file** containing the following:

- all related scripts/Jupyter notebooks,
- model weights for each model trained[2],
- a 'requirements.txt',
- a short README file explaining how to run scripts,
- your final paper in PDF format generated using the LaTeX template specified above;

and **(ii) a short video** (5 min maximum) describing your work and report with a walkthrough of your Jupyter notebook (see above for further details). Marks will be deducted if video file is missing. *Do not upload your dataset*, we have this! Any feature extraction or re-writing of data into other formats should all be reproducible within the code. The submission **deadline is: 9:00am, 17th of December 2025**.

## 4.2 Plagiarism

All submissions will be checked for plagiarism. Please review the School's plagiarism policy here. Any suspected plagiarism will be dealt with accordingly.

**You are not permitted to use AI tools** such as ChatGPT, GitHub Copilot, or similar for any part of this project expect for dataset augmentation.

---

# 5 Grading guidelines

You find below a set of guidelines relating to the project evaluation. These are indicative and non-binding.

## 5.1 Code (40 marks)

*Algorithmic correctness (10 marks)*

- Breadth & depth of investigations e.g. implementation of various models...
- Model output making sense.
- Implementation matching the description in the paper...

*Experimental correctness (10 marks)*

- Quality of implementation.
- Correctness of methodology: appropriate use of training, test and validation set (split with seeds if not split already or according to instructions), data shuffling, correct pre-processing...
- Searching hyperparameter space: fair hyper-parameter tuning across all algorithms.

*Reproducibility (10 marks)*

- Inclusion of README.
- Inclusion of clear comments and (package) requirements.txt.
- Data processing reproducibility i.e. can it be run all over again automatically?
- Model reproducibility: if models are run again on another machine fulfilling requirements, roughly the same performance will be achieved[3].

*Modelling and Innovation (10 marks)*

- Algorithmic novelty and/or experimental novelty.
- Use of several models in comparison.
- Transfer learning if appropriate.
- Interesting data pre-processing.
- Exploring the use of raw data...etc – essentially anything *'interesting'* !

## 5.2 Report (60 marks)

Reports will be graded based on correctness and clarity as follows:

- Background research, Related work and References (10 marks)
- Experimental setup (15 marks)
- Results section (15 marks)
- Conclusion & Future work (5 marks)
- Evaluation & Achievements (10 marks)
- Literacy & clarity, Quality of illustrations, Graphs and tables (5 marks)

[2] Use either the standard PyTorch format or H5 format.

[3] ± some variance since models are randomly initialised.

### 5.3 Video (30 mark deduction if missing)

- MP4 format using H.264 codec
- Screen capture with embedded video of yourself
- See above for further details

---

## 6 Resources

### 6.1 Report: section content

- *Abstract:* General, short overview of your paper (draw in the readers attention).
- *Introduction:* Introduce the problem in more detail, the motivation for your work, a brief description of your approach and findings along with a description of how the rest of the paper will proceed.
- *Related work:* A short summary of past approaches to this problem, the state-of-the-art (if available). You should compare and contrast your own work to these existing works. Referenced results and conclusions should be used to motivate your experiments.
- *Experimental setup:* A detailed description of the dataset used, preprocessing applied (if any), proposed algorithm, baseline approach, hyperparameter tuning done...etc. From this section, the reader should understand exactly what you have done without reading your code.
- *Results:* Explain evaluation technique and motivation behind using it (i.e. Accuracy, MSE, F1-score,...etc.) Compare your approach to baselines (i.e. a simpler model) or state of the art.
- *Conclusion & Future work:* An overview of your main findings. You should also propose how this model might be improved in future (i.e. what would you do if you had more time or could do the project again?).
- *References:* Reference any papers that you used in your related work or as a reference for your approach.

### 6.2 DL Frameworks and LaTeX

**Pytorch:**

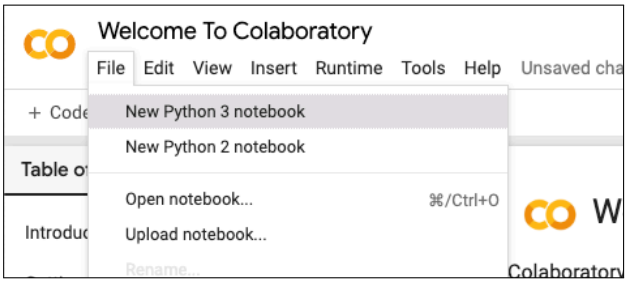
- *Download:* `https://pytorch.org/`
- *Tutorials:* `https://pytorch.org/tutorials/`
- *Forum:* `https://discuss.pytorch.org/`

**LaTeX:**

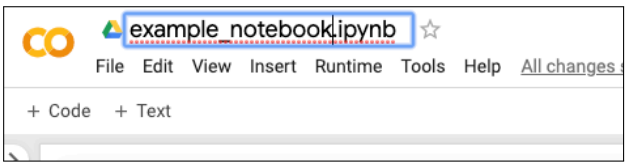
- *Online compiler:* `https://www.overleaf.com/`
- *Tutorials:* `https://www.overleaf.com/learn/latex/Tutorials`

### 6.3 Google Colab

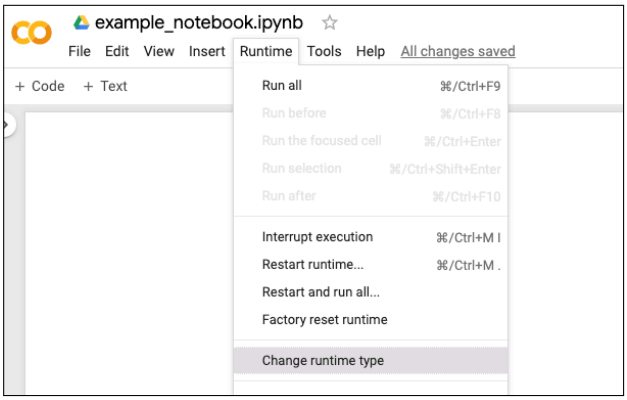
- Create new notebook



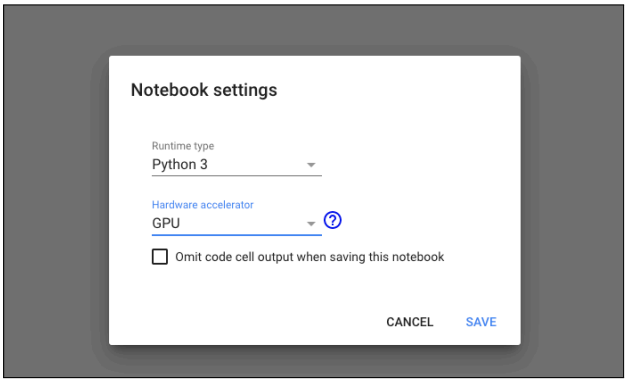

- Rename notebook



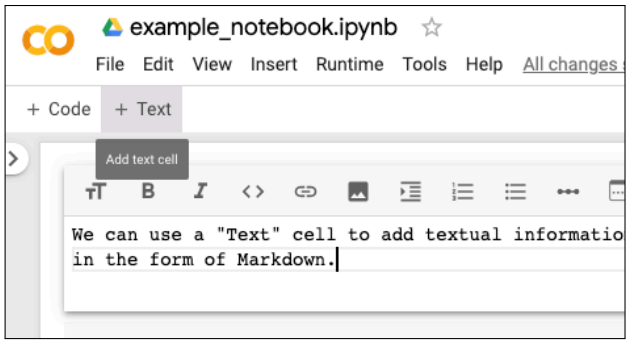

- Change runtime in order to switch between CPU, GPU and TPU



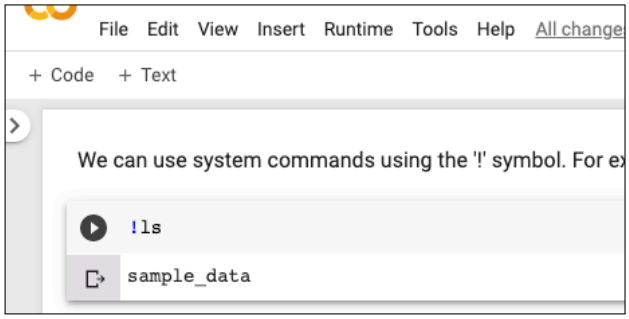

Select either None, GPU or TPU



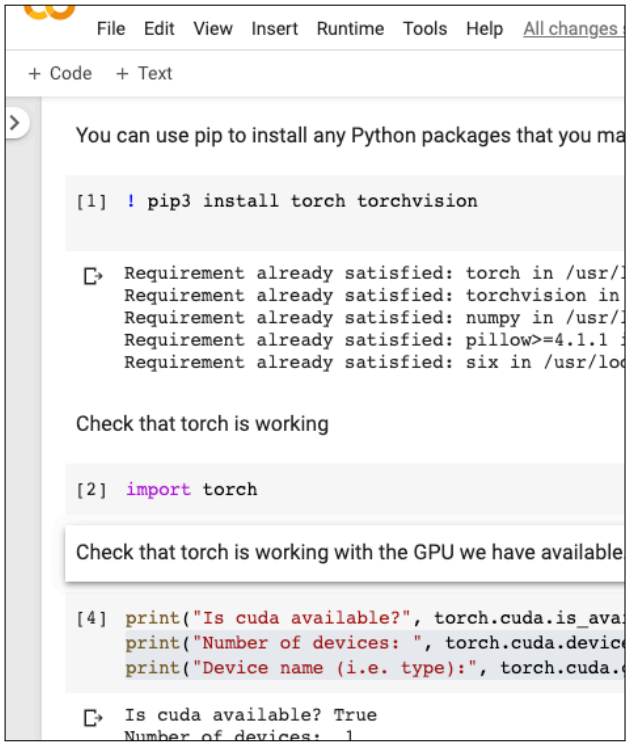

- Write details about code or additional information using the text cells.



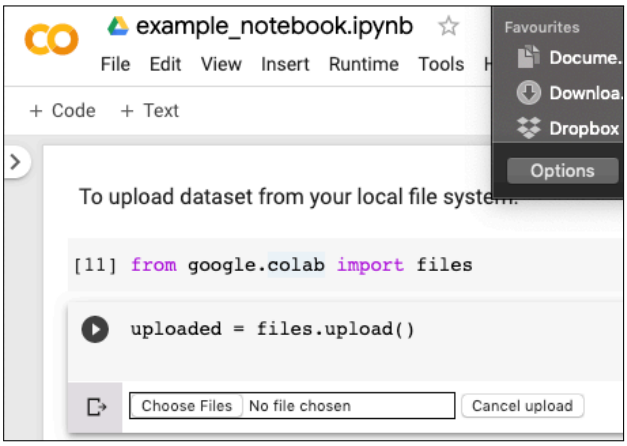

– You can use system commands (i.e. linux commands) using the '!' symbol in code cells.

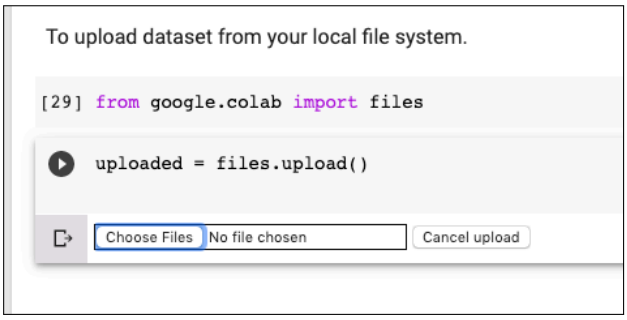


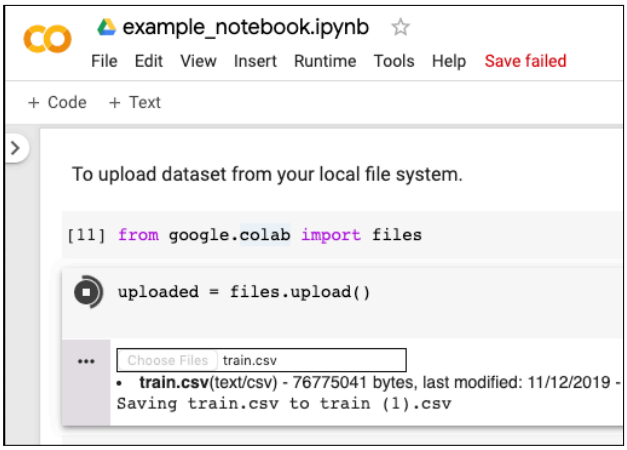

– Installing Pytorch and using it with a GPU.




– Uploading data from your local disk.




– You can connect your Colab notebook to your google drive: `https://colab.research.google.com/drive/`

– You can also use the kaggle API to download data: `https://www.kaggle.com/docs/api`

---

oOo